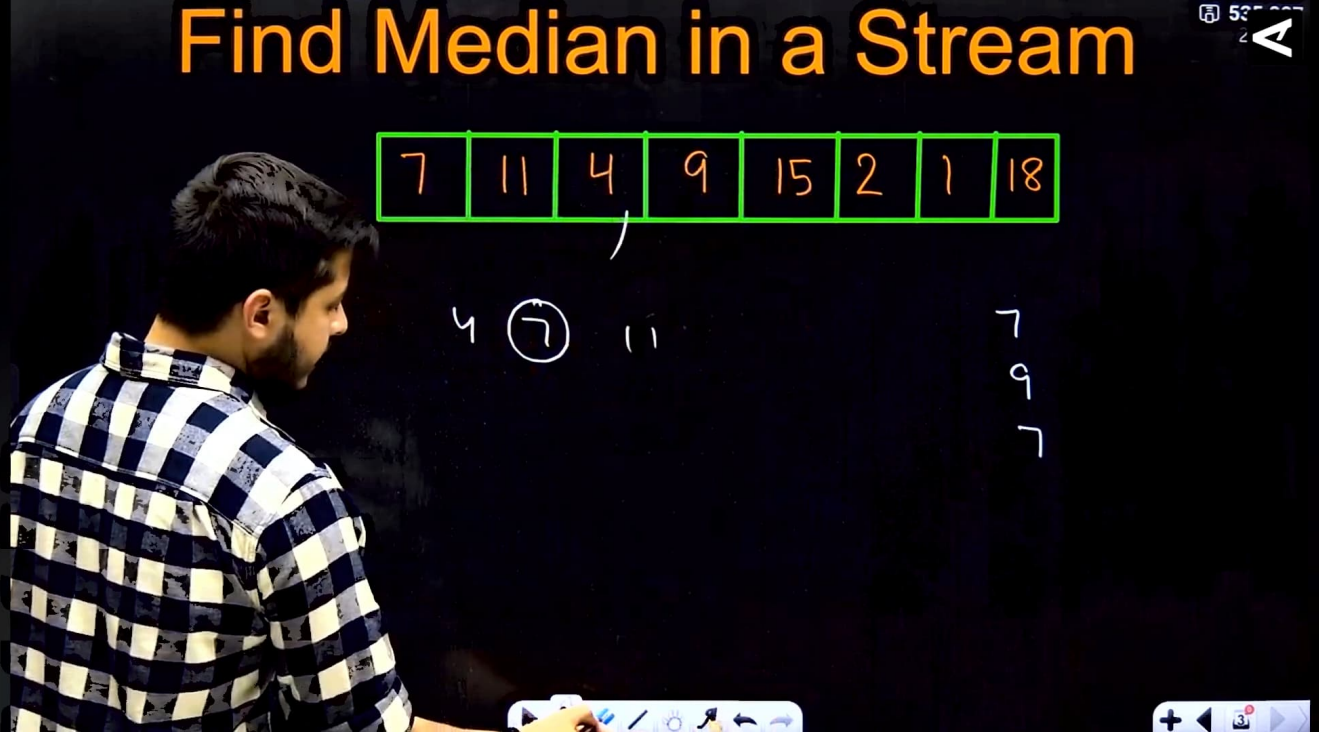
Find Median in a Stream



Find Median in a Stream

4 7 9 11 15

7
9
7
8
9

Find Median in a Stream

7	11	4	9	15	2	1	18

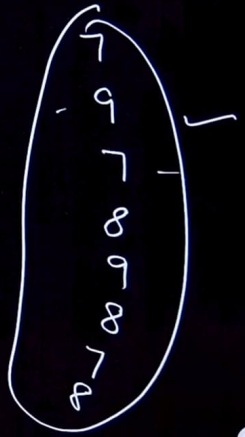
4 7 9 11 15 18

7 9 7 8 9 8 7 8

Find Median in a Stream

जी और उनको मैं उनकी सही पोजीशन पे लगा रहा हूं तो ऐसा

Find Median in a Stream



तो यहां पे क्या हो रहा है शिफ्टिंग हो रही है क्या हो रही

Find Median in a Stream

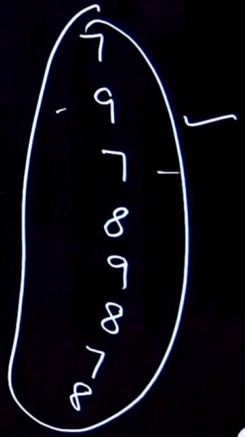
7	11	4	9	15	2	1	18

2	4	7	9	11	15

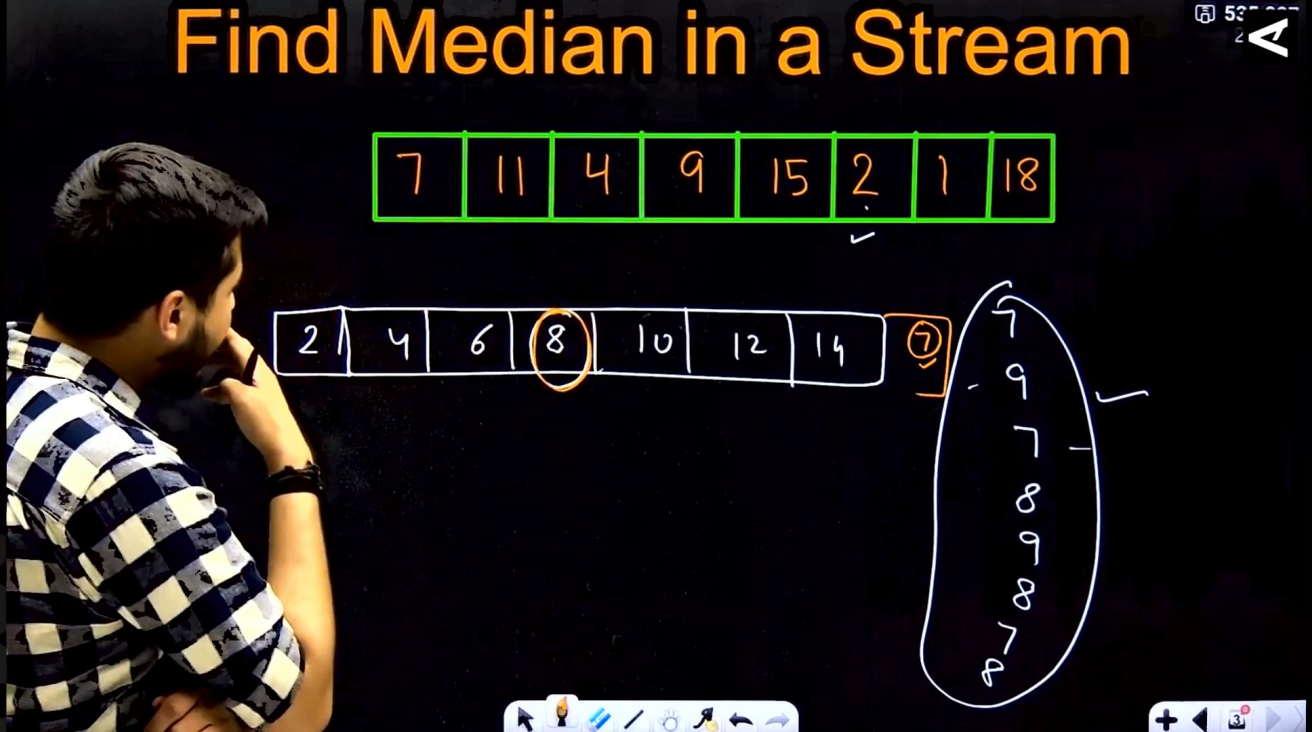
$0 + 1 + 2 + 3 \cdots n-1$

$\Downarrow$

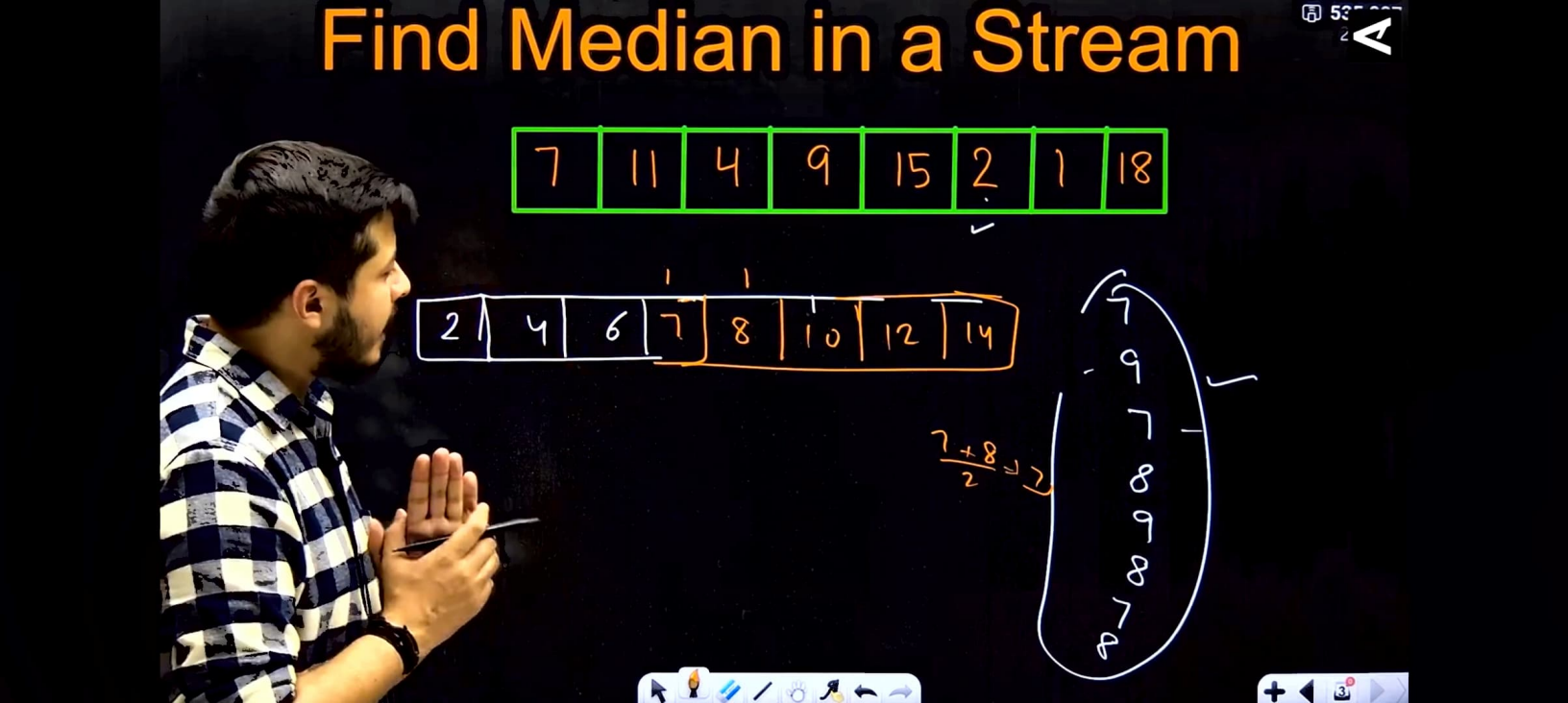
$O(N^2)$



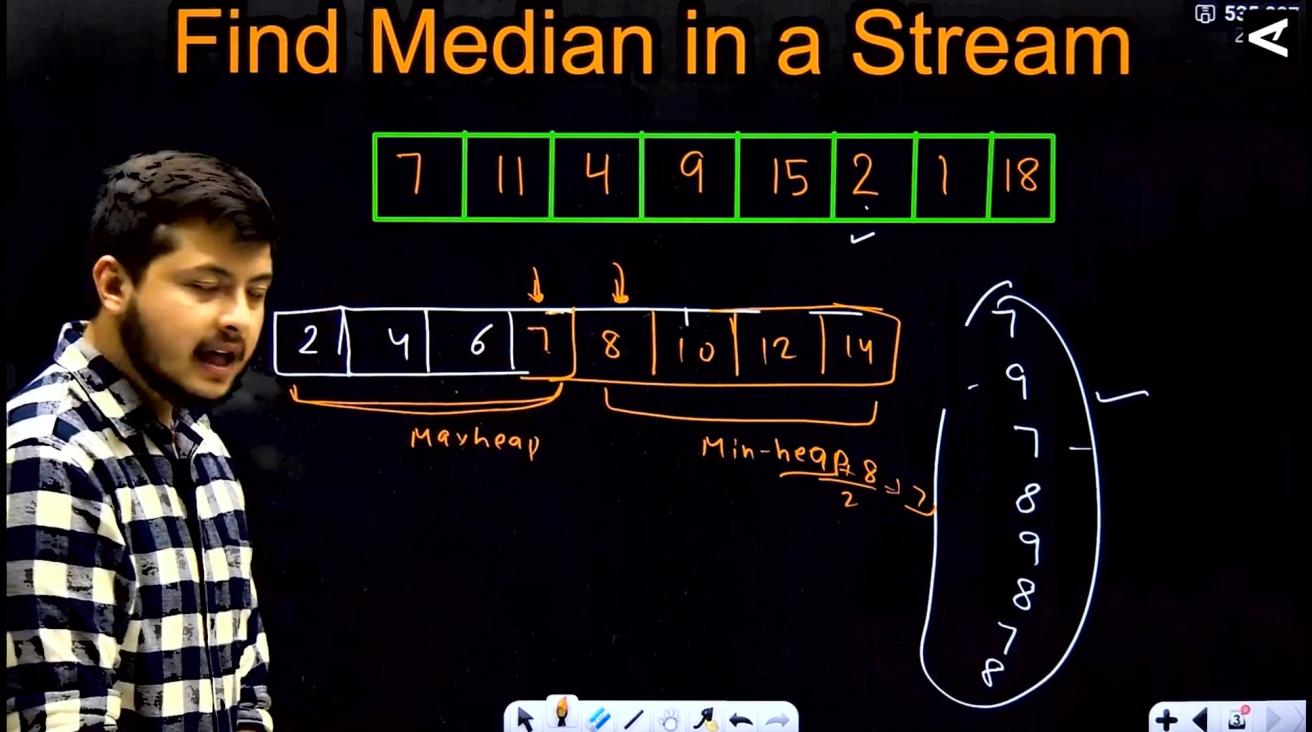
Find Median in a Stream



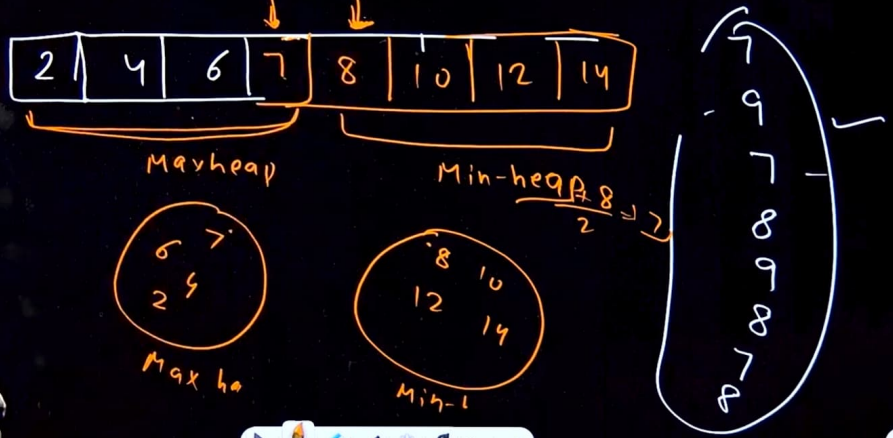
Find Median in a Stream



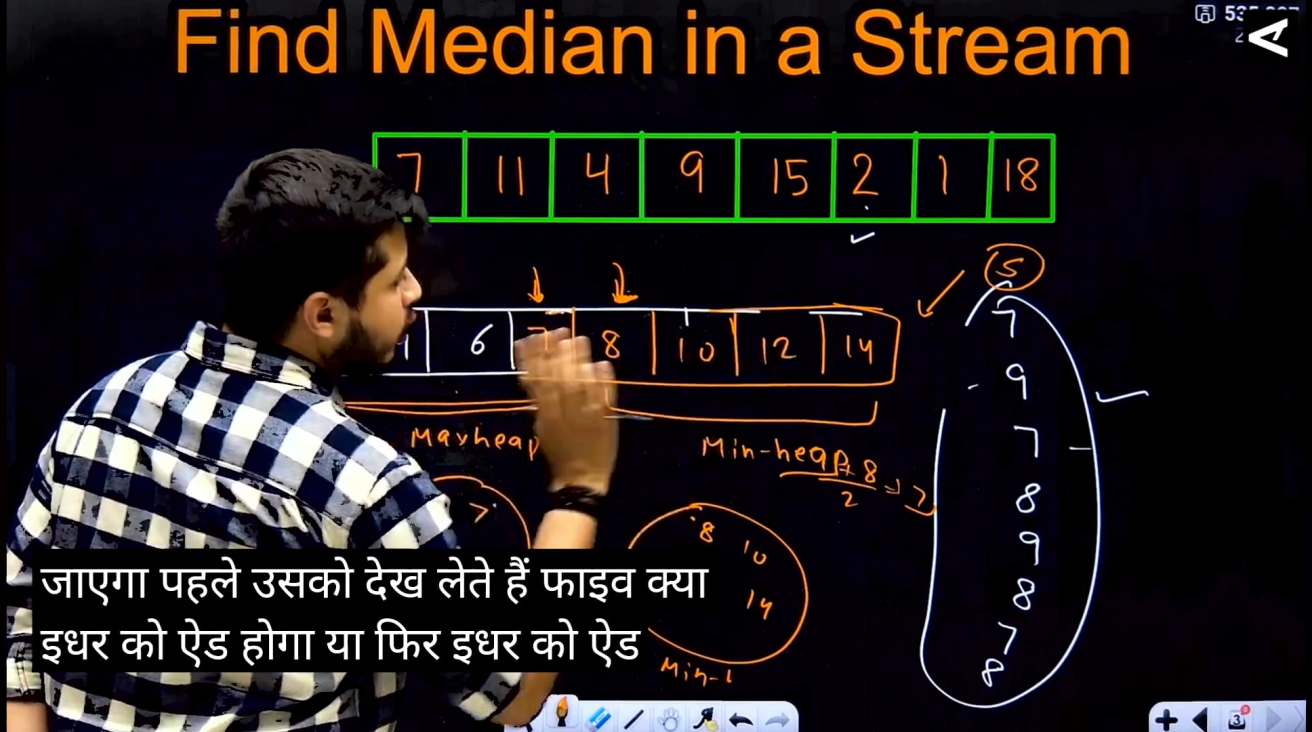
Find Median in a Stream



Find Median in a Stream

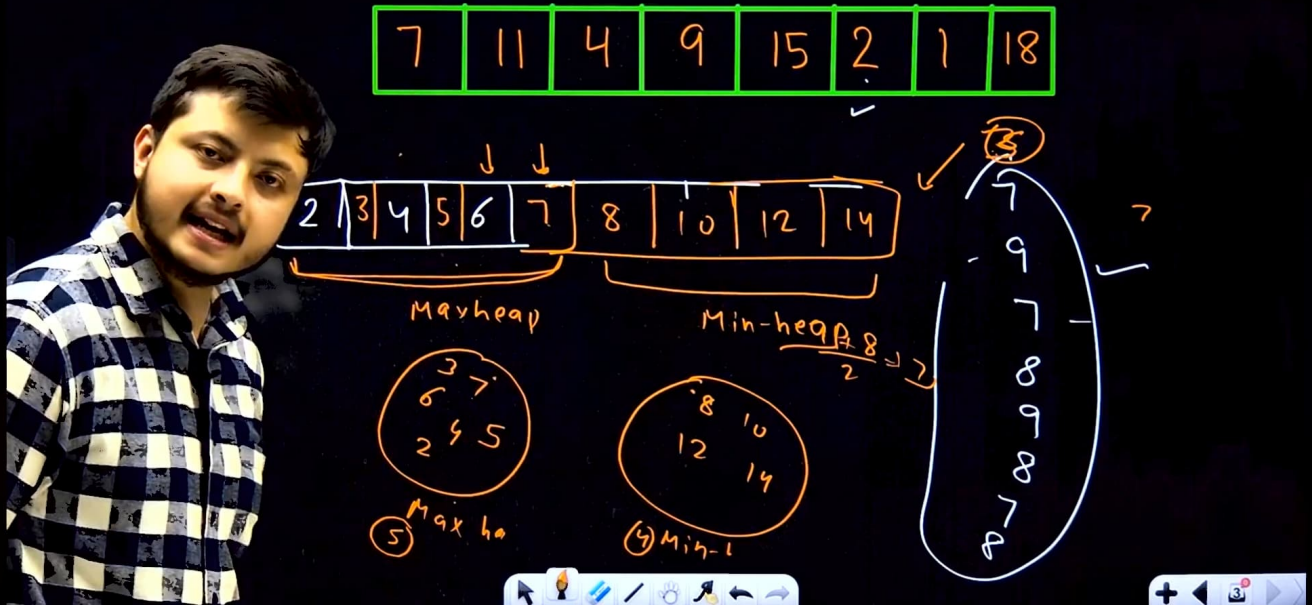


Find Median in a Stream

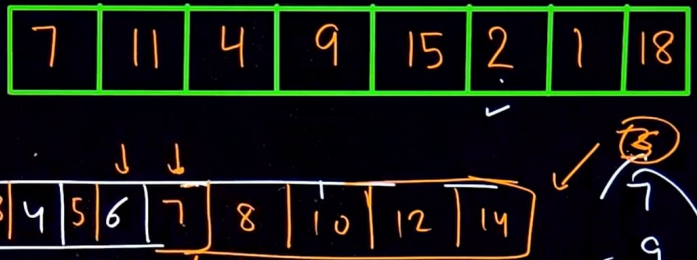


जाएगा पहले उसको देख लेते हैं फाइव क्या
इधर को ऐड होगा या फिर इधर को ऐड

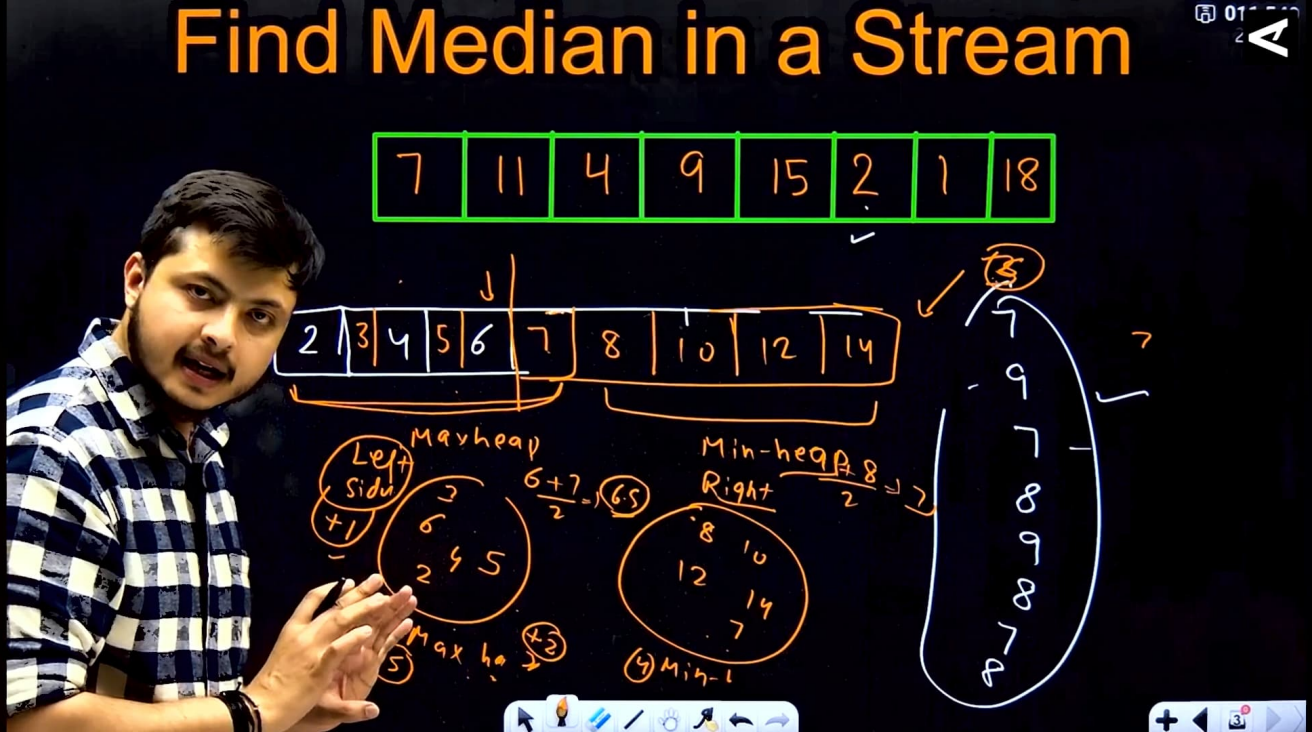
Find Median in a Stream

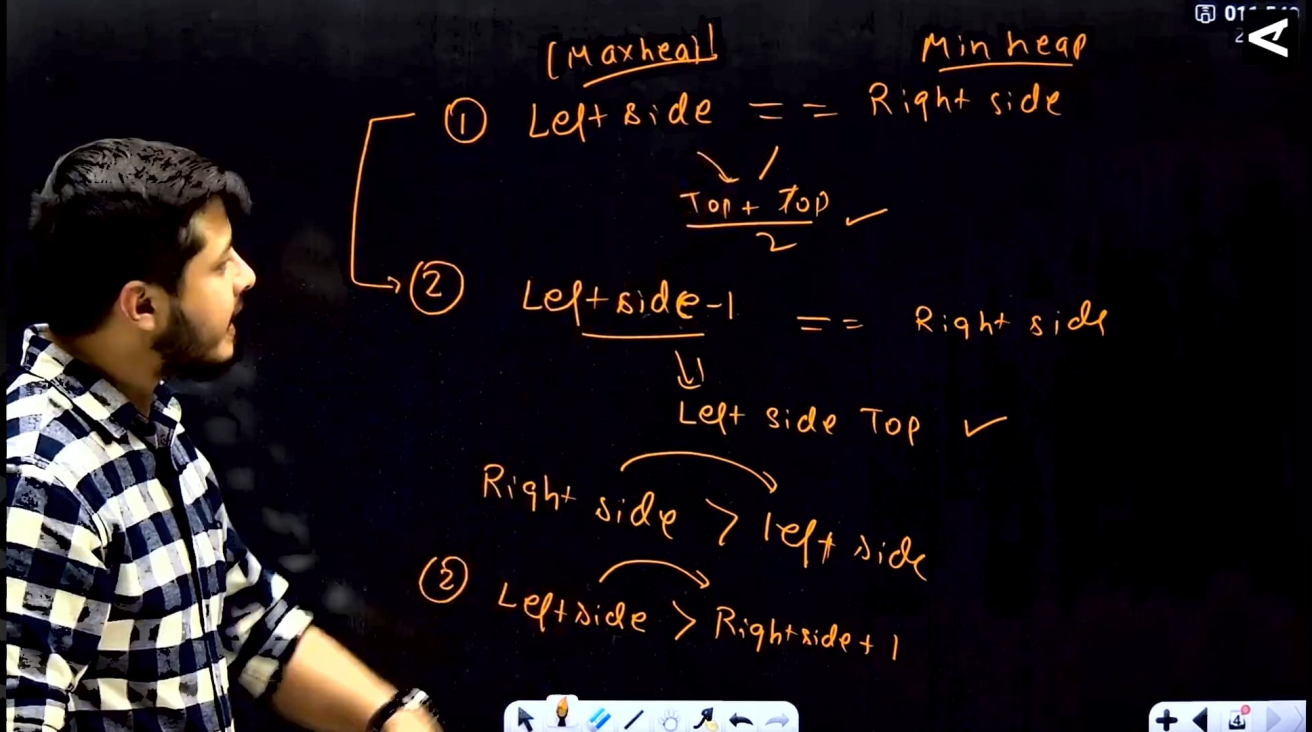


Find Median in a Stream



Find Median in a Stream




[Maxheap]
Min heap
① Left side == Right side
Top + Top
2
② Left side - 1 == Right side
Left side Top
Right side > left side
② Left side > Right side + 1

Problem | Editorial | Submissions | Comments

Find median in a stream

Hard Accuracy: 30.34% Submissions: 115K+ Points: 8

Given an input stream of **N** integers. The task is to insert these numbers into a new stream and find the median of the stream formed by each insertion of **X** to the new stream.

Example 1:

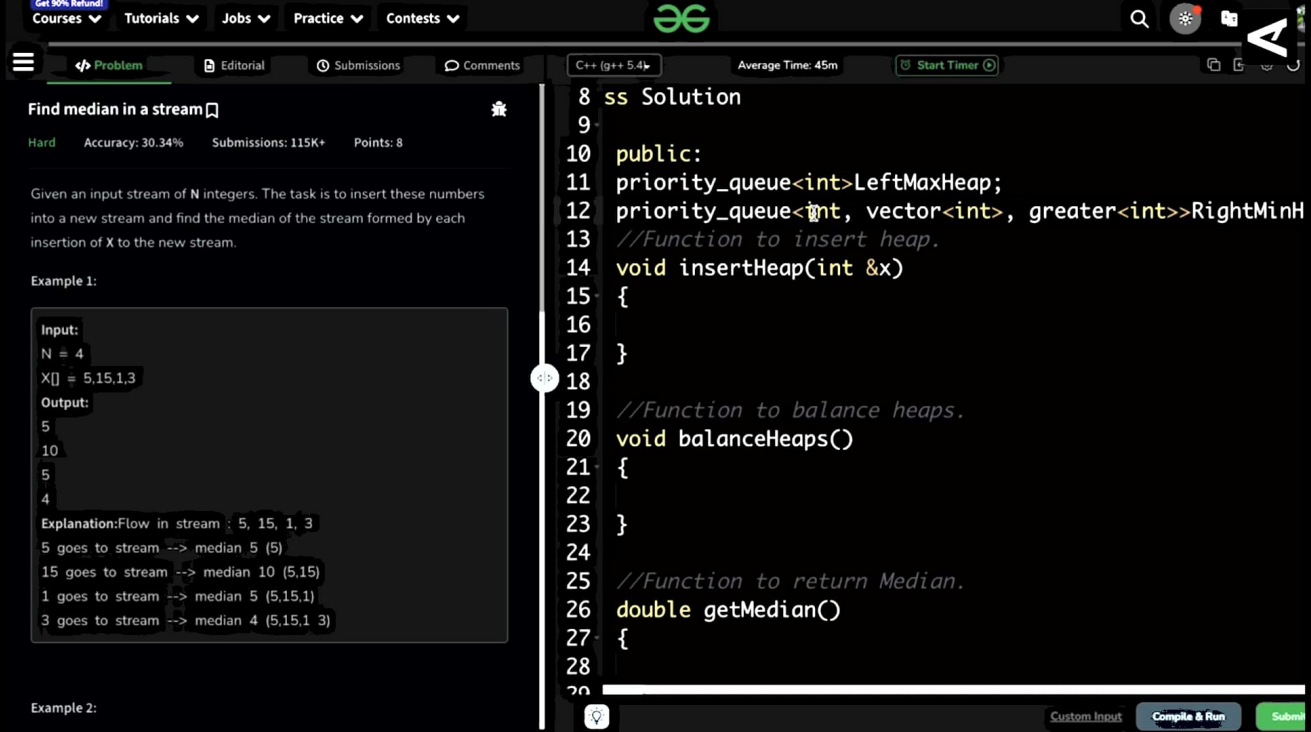
Input:
N = 4
X[] = 5,15,1,3
Output:
5
10
5
4
Explanation: Flow in stream : 5, 15, 1, 3
5 goes to stream --> median 5 (5)
15 goes to stream --> median 10 (5,15)
1 goes to stream --> median 5 (5,15,1)
3 goes to stream --> median 4 (5,15,1 3)

Example 2:

C++ (g++ 5.4) | Average Time: 45m | Start Timer

```
{
    public:

    //Function to insert heap.
    void insertHeap(int &x)
    {

    }

    //Function to balance heaps.
    void balanceHeaps()
    {

    }

    //Function to return Median.
    double getMedian()
    {

    }
};
```

Custom Input | Compile & Run


Get 90% Refund!
Courses
Tutorials
Jobs
Practice
Contests
Problem
Editorial
Submissions
Comments
C++ (g++ 5.4)
Average Time: 45m
Start Timer
Find median in a stream
Hard
Accuracy: 30.34%
Submissions: 115K+
Points: 8
Given an input stream of N integers. The task is to insert these numbers into a new stream and find the median of the stream formed by each insertion of X to the new stream.
Example 1:
Input:
N = 4
X[] = 5,15,1,3
Output:
5
10
5
4
Explanation:Flow in stream : 5, 15, 1, 3
5 goes to stream --> median 5 (5)
15 goes to stream --> median 10 (5,15)
1 goes to stream --> median 5 (5,15,1)
3 goes to stream --> median 4 (5,15,1 3)
Example 2:
ss Solution
public:
priority_queue<int>LeftMaxHeap;
priority_queue<int, vector<int>, greater<int>>RightMinH
//Function to insert heap.
void insertHeap(int &x)
{
}
//Function to balance heaps.
void balanceHeaps()
{
}
//Function to return Median.
double getMedian()
{
Custom Input
Compile & Run
Submi

10

Explanation: Flow in stream : 5, 10, 15
5 goes to stream --> median 5 (5)
10 goes to stream --> median 7.5 (5,10)
15 goes to stream --> median 10 (5,10,15)

Your Task:

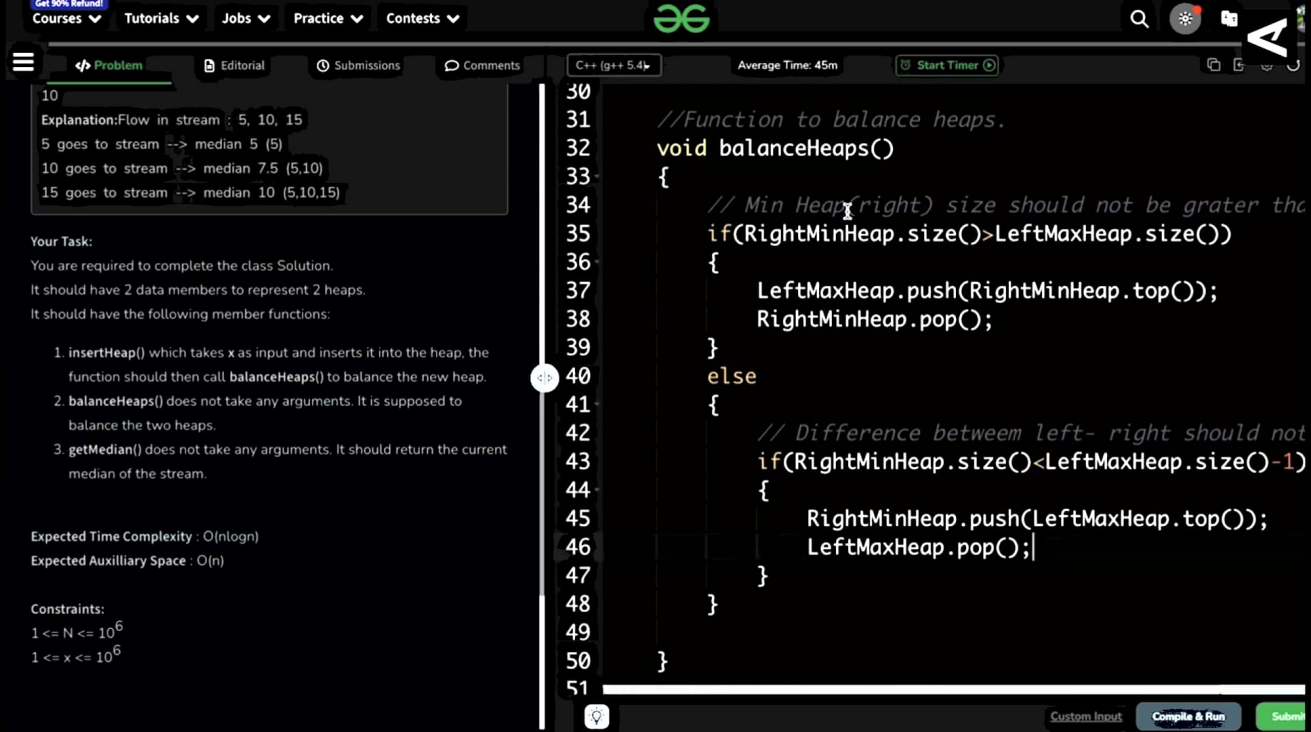
You are required to complete the class Solution.
It should have 2 data members to represent 2 heaps.
It should have the following member functions:

1. **insertHeap()** which takes **x** as input and inserts it into the heap, the function should then call **balanceHeaps()** to balance the new heap.
2. **balanceHeaps()** does not take any arguments. It is supposed to balance the two heaps.
3. **getMedian()** does not take any arguments. It should return the current median of the stream.

Expected Time Complexity : O(nlogn)
Expected Auxilliary Space : O(n)

Constraints:
$1 <= N <= 10^6$
$1 <= x <= 10^6$

```
17          if(LeftMaxHeap.empty())
18          {
19              LeftMaxHeap.push(x);
20              return;
21          }
22
23          if(x>LeftMaxHeap.top())
24          RightMinHeap.push(x);
25          else
26          LeftMaxHeap.push(x);
27
28          balanceHeaps;
29
30      }
31
32      //Function to balance heaps.
33      void balanceHeaps()
34      {
35
36      }
37
```


Courses
Tutorials
Jobs
Practice
Contests
Problem
Editorial
Submissions
Comments
C++ (g++ 5.4)
Average Time: 45m
Start Timer
10
Explanation:Flow in stream : 5, 10, 15
5 goes to stream --> median 5 (5)
10 goes to stream --> median 7.5 (5,10)
15 goes to stream --> median 10 (5,10,15)
Your Task:
You are required to complete the class Solution.
It should have 2 data members to represent 2 heaps.
It should have the following member functions:
1. insertHeap() which takes x as input and inserts it into the heap, the function should then call balanceHeaps() to balance the new heap.
2. balanceHeaps() does not take any arguments. It is supposed to balance the two heaps.
3. getMedian() does not take any arguments. It should return the current median of the stream.
Expected Time Complexity : O(nlogn)
Expected Auxilliary Space : O(n)
Constraints:
1 <= N <= 10^6
1 <= x <= 10^6
30
31 //Function to balance heaps.
32 void balanceHeaps()
33 {
34 // Min Heap(right) size should not be grater tha
35 if(RightMinHeap.size()>LeftMaxHeap.size())
36 {
37 LeftMaxHeap.push(RightMinHeap.top());
38 RightMinHeap.pop();
39 }
40 else
41 {
42 // Difference betweem left- right should not
43 if(RightMinHeap.size()<LeftMaxHeap.size()-1)
44 {
45 RightMinHeap.push(LeftMaxHeap.top());
46 LeftMaxHeap.pop();
47 }
48 }
49
50 }
51
Custom Input
Compile & Run
Submi

Problem Editorial Submissions Comments

10

Explanation: Flow in stream : 5, 10, 15
5 goes to stream --> median 5 (5)
10 goes to stream --> median 7.5 (5,10)
15 goes to stream --> median 10 (5,10,15)

Your Task:

You are required to complete the class Solution.
It should have 2 data members to represent 2 heaps.
It should have the following member functions:

1. **insertHeap()** which takes **x** as input and inserts it into the heap, the function should then call **balanceHeaps()** to balance the new heap.
2. **balanceHeaps()** does not take any arguments. It is supposed to balance the two heaps.
3. **getMedian()** does not take any arguments. It should return the current median of the stream.

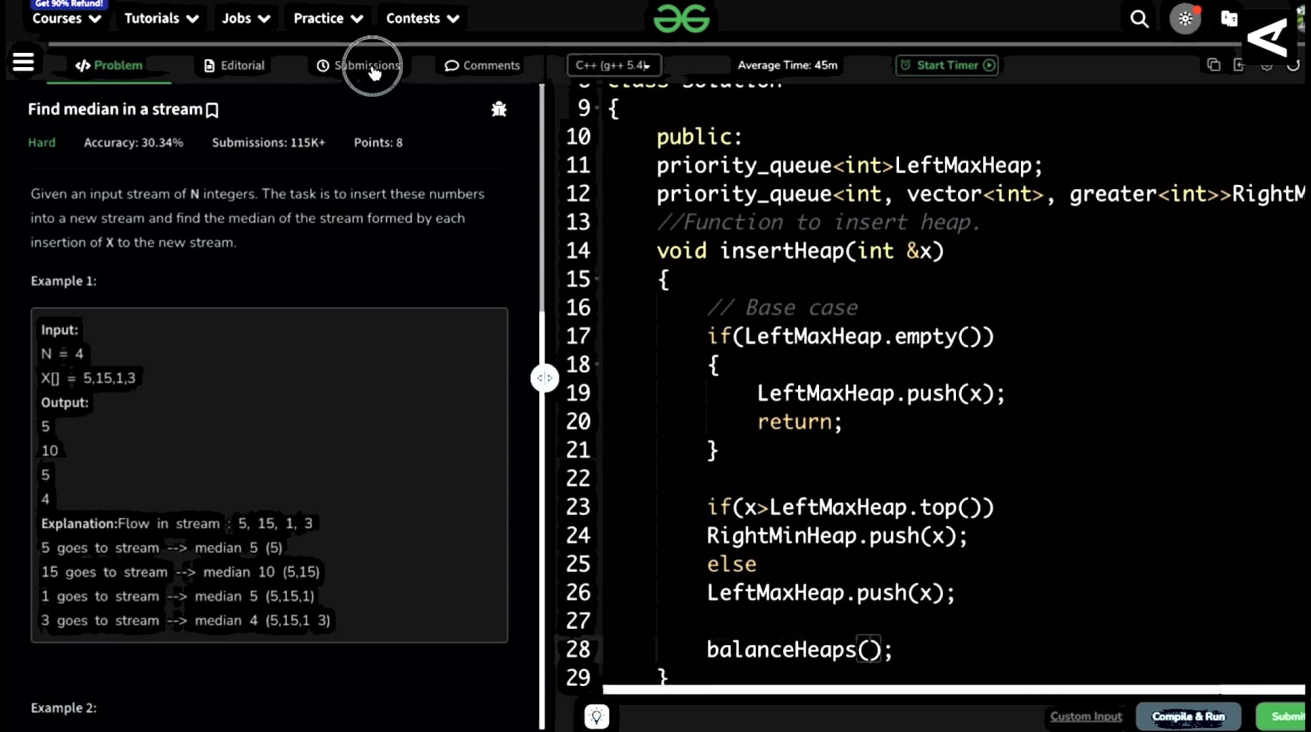
Expected Time Complexity : O(nlogn)
Expected Auxilliary Space : O(n)

Constraints:
$1 <= N <= 10^6$
$1 <= x <= 10^6$

C++ (g++ 5.4) Average Time: 45m Start Timer

```
46                  LeftMaxHeap.pop();
47              }
48          }
49
50      }
51
52      //Function to return Median.
53      double getMedian()
54      {
55          // Left > right
56          if(LeftMaxHeap.size()>RightMinHeap.size())
57          return LeftMaxHeap.top();
58          else
59          {
60              double ans = LeftMaxHeap.top()+ RightMinHeap
61              ans/=2;
62              return ans;
63          }
64      }
65  };
66
67
```

Custom Input Compile & Run Submi


Courses
Tutorials
Jobs
Practice
Contests
Problem
Editorial
Submissions
Comments
C++ (g++ 5.4)
Average Time: 45m
Start Timer
Find median in a stream
Hard
Accuracy: 30.34%
Submissions: 115K+
Points: 8
Given an input stream of N integers. The task is to insert these numbers into a new stream and find the median of the stream formed by each insertion of X to the new stream.
Example 1:
Input:
N = 4
X[] = 5,15,1,3
Output:
5
10
5
4
Explanation:Flow in stream : 5, 15, 1, 3
5 goes to stream --> median 5 (5)
15 goes to stream --> median 10 (5,15)
1 goes to stream --> median 5 (5,15,1)
3 goes to stream --> median 4 (5,15,1 3)
Example 2:
{
public:
priority_queue<int>LeftMaxHeap;
priority_queue<int, vector<int>, greater<int>>RightM
//Function to insert heap.
void insertHeap(int &x)
{
// Base case
if(LeftMaxHeap.empty())
{
LeftMaxHeap.push(x);
return;
}
if(x>LeftMaxHeap.top())
RightMinHeap.push(x);
else
LeftMaxHeap.push(x);
balanceHeaps();
}
Custom Input
Compile & Run
Submi

Find Median in a Stream.

Median – is the middle value in an ordered dataset.

for 7 11 4 => 4 [7] 11

& for 7 11 4 9

=> 4 [7 9] 11

$(7+9)/2 = 8$

7	11	4	9	15	2	1	18

Unordered	Ordered	Median
7	7	7
7 11	7 11	9
7 11 4	4 7 11	7
7 11 4 9	4 [7 9] 11	8
7 11 4 9 15	4 7 [9] 11 15	9

return this vector as ans.

1st: Brute-force Approach:

Either, with every coming element,

i) we sort the arr & find the median, T.C: $N(N \log N)$ ← for sorting

OR

ii) we place the new ele to it's correct position to maintain the sorted order.

$N * (N)$ ← for shifting

T.C: $O(N^2)$

FC $\simeq O(N^2 \log N)$

Approach 2:

We basically want our data to be in sorted form & when the new ele comes, we can simply insert into our DS without disturbing the sorted order.

2	4	6	8	10	12	14

Now if 7 comes

2	4	6	7	8	10	12	14

maxheap | minheap

if we observe carefully,

Median = $\frac{(\text{top ele (minh)} + \text{top ele (maxh)})}{2}$

The idea is to divide the data into 2 equal halves within minh & maxh in a way, their top element will be the median.

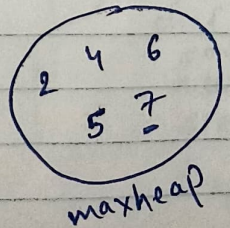
minheap.size() == maxheap().size() $7+8/2$

median = $\frac{\text{topEle(minh)} + \text{topEle(maxh)}}{2}$

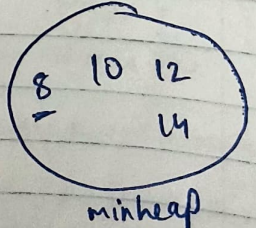
& if maxheap.size() > minheap.size or

maxheap.size() = minheap.size() + 1

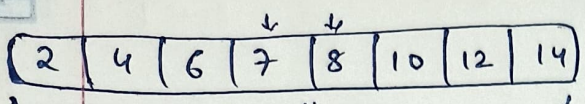
median = topEle(maxh). => 7



maxheap

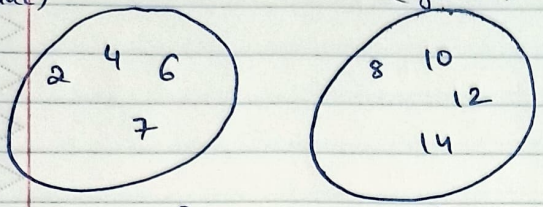


minheap



maxheap (leftside)

minheap (rightside)



maxheap minheap.

3 functions to complete.

```
//func to insert heap.
void insertHeap(int &x){
....
}
// function to balance heaps
void balanceHeaps(){
...
}
// function to return median
double getMedian(){
...
}
```

we have to call → this 2 will be called by compiler.

```
// Declare 2 heaps at the top as it will
// be used by all 3 funcs.
public:
priority_queue<int> LeftMaxHeap;
priority_queue<int, vector<int>, greater<int>> RightMinHeap;
```

Psuedocode:

① Leftside (Maxheap) == RightSide (Minheap)

median = (top + top) / 2

② Leftside - 1 == Rightside

median = Leftside (top)

③ Rightside > Leftside

pop from rightside (Minheap)

& push into leftside (Maxheap)

until

Leftside == rightside

or

Leftside - 1 == rightside

if (Leftside - 1 > Rightside)

pop from leftside

& push into rightside

```
// function to insert heap.
void insertHeap(int &x) {
    // Base case
    if (LeftMaxHeap.empty()){
        LeftMaxHeap.push(x);
        return;
    }
    if (x > LeftMaxHeap.top())
        RightMinHeap.push(x);
    else
        LeftMaxHeap.push(x);
    balanceHeaps();
}
```

```
//function to balance heaps.
void balanceHeaps(){
// MinHeap(Right) size should not be greater
// than MaxHeap
if (RightMinHeap.size() > LeftMaxHeap.size())
{ LeftMaxHeap.push(RightMinHeap.top());
  RightMinHeap.pop();
}
else{
// only if the difference b/w
// left-right > 1
if(RightMinHeap.size() < LeftMaxHeap.size()
                          -1)
{ RightMinHeap.push(LeftMaxHeap.top());
  LeftMaxHeap.pop();
}
}
```

left − right > 1	left	right
	5	4
	6	4
		X

```
//func to return Median
double getMedian(){
// Left > right
if (LeftMaxHeap.size() > RightMinHeap.size())
    return LeftMaxHeap.top();
else{
    double ans = top(MinH) + top(MaxH);
    ans /= 2;
    return ans;
}
}
```